the probability tensor. We do this for all three actions and select the one that has the highest expected value. Once you get comfortable with the code here, you can try coming up with a more sophisticated policy, perhaps one that takes into consideration the variance (i.e., the confidence) of each action-value distribution.

7.7. Using distributional Q-learning to play Freeway

We're finally ready to use the Dist-DQN algorithm to play the Atari game Freeway. We don't need any other major functionality besides what we've already described. We will have a main Dist-DQN model and a copy—the target network to stabilize training. We will use an epsilon-greedy strategy with a decreasing epsilon value over epochs: with probability epsilon the action selection will be random, otherwise the action will be selected by the `get_action` function, which chooses based on the highest expected value. We will also use an experience replay mechanism, just like with an ordinary DQN.

We will also introduce a very basic form of *prioritized replay*. With normal experience replay, we store all the experiences the agent has in a fixed-size memory buffer, and new experiences displace old ones at random; then we randomly sample a batch from this memory buffer for training. In a game like Freeway, though, where almost all actions result in a –1 reward and we rarely get a +10 or –10 reward, the experience replay memory is going to be heavily dominated by data that all says basically the same thing. It's not very informative to the agent, and the truly significant experiences such as winning or losing the game get strongly diluted, significantly slowing learning.

To alleviate this problem, whenever we take an action that leads to a winning or losing state of the game (i.e., when we get a reward of –10 or +10), we add multiple copies of this experience to the replay buffer to prevent it from being diluted by all the –1 reward experiences. Hence, we *prioritize* certain highly informative experiences over other less informative experiences because we really want our agent to learn which actions lead to success or failure rather than just game continuation.